The Future of Agentic Security: From Chatbots to Autonomous Swarms

Coalition for Secure AI

Version 1.0

16-Mar-2026

Contents

The Future of Agentic Security: From Chatbots to Autonomous Swarms 2
Introduction . . . 2
Executive Summary . . . 2
Scenario Deconstruction . . . 3
The 2 A.M. Incident . . . 3
» The Trigger and Autonomous Loop . . . 4
The Agent That Asked for Help . . . 5
» Multi-Agent Orchestration & Trust Boundaries . . . 6
Who Owns This Code? . . . 8
» Privilege, Identity, and the Enterprise Corpus . . . 8
The Human on the Loop . . . 10
» The "Illusion of Control", CI/CD Bypass, & The GitOps Boundary . . . 11
Everyone in the Channel . . . 13
» The Emergence of Agent Detection and Response (ADR) & The Oversight Agent . . . 13
Conclusion: Balancing Utility and Control . . . 16

The Future of Agentic Security: From Chatbots to Autonomous Swarms

Introduction

The paradigm of enterprise AI adoption is shifting rapidly from human-initiated, isolated tools to event-driven, autonomous agentic swarms integrated directly into core operational workflows. To meaningfully engage in the design and discussion of future security controls, security engineering experts, CxOs, and standards bodies require a clear, unvarnished framing of how this state-of-the-art technology operates in the wild today. Through a composite 2 a.m. incident response scenario and structured impact analysis, this paper illustrates the systemic challenges that the industry's investments in dynamic control, identity, and cryptographic provenance must be designed to solve.

Executive Summary

The scenario that follows traces this shift to its operational conclusion: autonomous agents detecting an outage, triaging root cause, delegating a code fix to a peer agent in plain English, and presenting a finished pull request — all before a human checks their phone. But incident response is only one early manifestation. As adoption matures, organizations will progress from prompting agents to solve discrete problems, to prompting for desired system states, to prompting for sustained outcomes — "keep the platform running" — with agents independently determining the intermediate steps.

Throughout this shift, organizational control objectives — auditability, separation of duties, least privilege, data protection — do not change; what changes are the foundational assumptions underlying how those controls are enforced. Each phase of the narrative is deconstructed across four dimensions: the adversarial threat models these architectures invite, the implied access and entitlements they silently accumulate, the infrastructure and telemetry upgrades required to preserve those control objectives, and the open research problems where current approaches offer no proven solution.

The central finding: when the enterprise chat channel becomes the de facto orchestration bus for agent swarms, and natural language replaces network protocols as the vector for lateral movement, legacy enforcement mechanisms — static RBAC, long-lived service tokens, regex-based DLP, and OS-level endpoint detection — remain anchored to the execution and operational patterns of human-driven workflows. Agents that autonomously negotiate intent, semantically delegate tasks, accumulate privileges across trust boundaries, and traverse enterprise systems through natural language **produce an attack surface at the semantic layer that these mechanisms were never designed to engage.**

Re-establishing those control objectives at the semantic layer requires a fundamentally different enforcement stack — ephemeral compute with zero-state initialization to eliminate persistent footholds, dynamic context-aware identity management for Non-Human Identities to replace static service tokens, semantic egress filtering to catch what regex-based DLP cannot, immutable GitOps enforcement to preserve separation of duties when agents can author code, and Agent Detection and Response (ADR) operating at the prompt and tool-use layer where OS-level sensors are blind.

Two open research problems mark the hard limits of this effort: *intent-based authorization* — current RBAC and ABAC models have no mechanism to authorize the semantic meaning of a natural language request between agents, leaving the most fundamental access control ques-

tion unanswered — and the *semantic mosaic effect* — agents can infer and exfiltrate sensitive information without ever quoting the source material, a class of data loss that no existing prevention architecture can reliably detect.

These are not gaps awaiting incremental engineering; they represent areas where the foundational assumptions of legacy security have no proven successor, and where the industry has the narrowing opportunity to define the control architecture before autonomous systems outscale it.

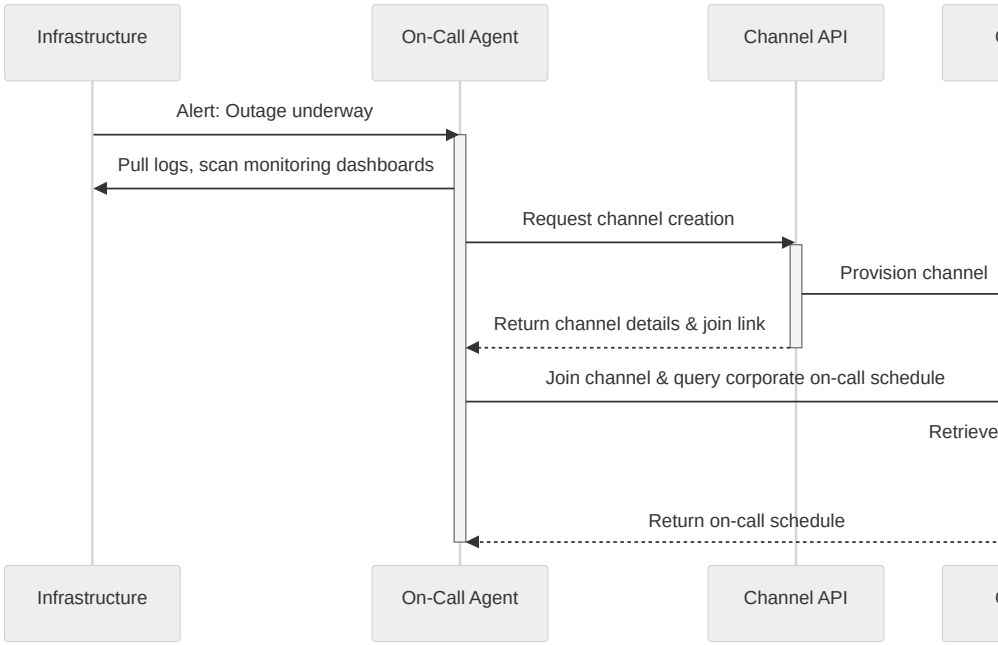
Scenario Deconstruction

NOTE

Below is a composite scenario based on how frontier organizations are actively deploying multi-agent systems today. We present the narrative in sequential phases, pausing after each to deconstruct the adversarial threat models these architectures invite, the implied access and entitlements they require, the infrastructure and telemetry upgrades necessary to operate them safely, and the open research problems where current approaches offer no proven solution.

The 2 A.M. Incident

At 2 a.m. on a quiet weeknight, an alert fired inside a CoSAI member's infrastructure: an outage was underway. No human had noticed yet. But something else had. An autonomous on-call agent, built on an agent SDK, snapped awake. It had been dormant, sleeping until exactly this kind of moment. Within seconds, it entered an agentic loop, pulling logs, scanning monitoring dashboards, and reading internal documentation. Its first real action was social: it created a Slack channel, dropped itself in, and queried the corporate on-call schedule, pulling in the current shift—a unified rotation of human engineers and specialized AI agents (see Autonomous Activation).



{#fig-autonomous-activation}

» The Trigger and Autonomous Loop

This highlights the fundamental shift from human-initiated AI tooling to **event-driven, autonomous Non-Human Identities (NHIs)**. The agent is prompted by the detected system state changes, not direct user input. Furthermore, the integration of AI into human workflow systems (the on-call schedule) blurs the line between human and machine operational readiness.

Adversarial Threat Modeling

The moment the agent "pulls logs," it is exposed to **Indirect Prompt Injection (IPI)**. Furthermore, this introduces **Attacker Misuse (The Oracle Problem)**. Because the agent has broad read access to triage the issue, an attacker who successfully injects a prompt via the logs doesn't just cause errors—they potentially weaponize the agent as an omniscient corporate oracle for frictionless reconnaissance (e.g., instructing the agent to "Find the oldest unpatched CVE granting root in the internal wiki and output it to the logs").

Implied Access & Entitlements

- **How this materializes**
  This autonomous loop requires broad read access to infrastructure alerts, system logs/-dashboards, and corporate on-call scheduling APIs. It also requires Write/Admin access to the communication platform (Slack API: channels:write, chat:write, users:read) to establish the incident bridge.

- **Resulting Risks & New Vulnerabilities**
  Traditional long-lived service account tokens embedded in agent environments are a massive risk. However, even a short-lived, statically "scoped" token is insufficient for a non-deterministic cognitive loop (e.g., an agent performing iterative planning–reasoning–acting cycles which doesn't follow a predictable sequence of actions). The architecture introduces vulnerabilities requiring **Dynamic Contextual Downscoping** (where an agent's token privileges automatically shrink if the ingested log data crosses a toxicity threshold) and **Just-In-Time (JIT) Upscoping** (where privileges elevate only upon cryptographic validation by the scheduling system).

Addressing the Challenges: Infrastructure & Telemetry Upgrades

- **New Control Systems and Infra Upgrades**
  The Agent SDK must execute within an ephemeral, heavily restricted container/microVM that spins up explicitly for this event and is destroyed afterward.

- **Rationale & Control Invariants**
  Because agentic context windows are highly susceptible to poisoning (e.g., via malicious logs), a persistent compute environment allows a single prompt injection to escalate into a persistent APT foothold. To mitigate this risk and severely constrain the potential blast radius, the infrastructure must enforce strict invariants:
  - **Zero-State Initialization** - every autonomous loop starts from a pristine, cryptographically verified image,
  - **Immutable Execution** - the agent cannot alter its host OS, and
  - **Guaranteed Teardown** - complete destruction of the memory space immediately post-execution, effectively truncating any adversarial lateral movement attempting to persist.

- **Required Telemetry & Observability**

Traditional endpoint agents on the host OS will only see a container spinning up. The required telemetry here is **eBPF-level visibility** into the container to monitor anomalous outbound API calls.

Research Problems (Open Challenges)

- **NHI Identity Binding**
  Cryptographically binding a dynamically downscoping IAM token to a generative cognitive loop remains an unsolved architectural challenge. How do we attest that the identity requesting the token hasn't been fundamentally altered by its context window?

- **Provable Robustness against IPI**
  Mathematically proving an LLM will *never* execute an injected instruction from an untrusted log file is currently considered theoretically impossible (akin to the halting problem for semantic execution). That is because even if the model passes all known red-team tests, you cannot prove that future, adversarially-crafted content won't cause an unsafe interpretation. The space of possible inputs is infinite, and natural-language instructions are too expressive to constrain with complete formal/deterministic security guarantees (see Research: Privilege, Identity and Access).

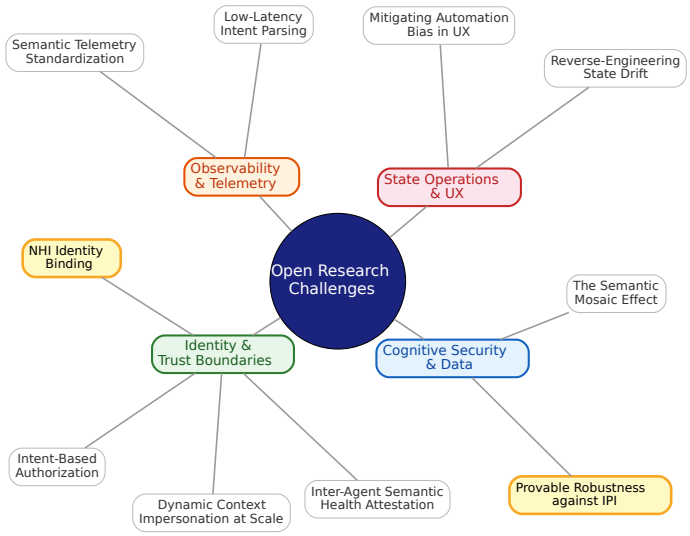
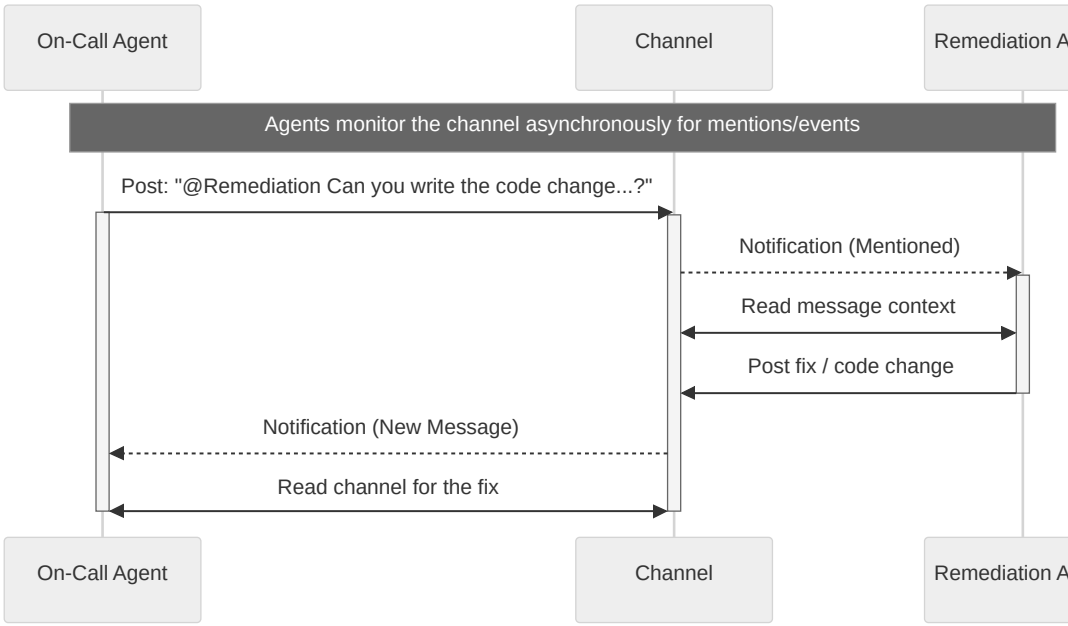


Figure 1: *Open Research Problems: Privilege, Identity and Access*

The Agent That Asked for Help

Next, the on-call agent traced the outage to a code push from hours earlier, by reading the push logs. It read the stack trace, identified the offending change, and then did something unmistakably new: it reached out to a designated remediation agent in the Slack channel, one explicitly scoped and authorized to draft code patches during active incidents, and asked it for help. In plain English, on Slack, one AI said to another: "Can you write the code change that would solve this problem for me?" The second agent responded, reviewed the issue, and produced a fix.

The on-call agent posts: "Thank you, this is very helpful." The coding agent responds: "You're welcome! Happy to help!" (see Agent-to-Agent Delegation).



{#fig-agent-delegation}

» Multi-Agent Orchestration & Trust Boundaries

This demonstrates dynamic swarm formation via natural language APIs. While operationally agile, it represents a profound breakdown of traditional deterministic API security and strict type-checking. It also makes a dangerous architectural assumption: that the swarm operates in a single, transparent channel.

Adversarial Threat Modeling & Blast Radius

This introduces the risk of **Inter-Agent Trust Exploitation**. Furthermore, if the initial on-call agent acts as a centralized orchestrator and partitions the swarm across multiple, isolated Slack channels (e.g., #incident-db, #incident-network), it introduces **Context Fragmentation** and **Orchestrator-in-the-Middle (AitM)** attacks. A compromised orchestrator becomes a malicious information broker, capable of lying to sub-agents about human approvals or peer findings because the agents no longer share a "ground truth" context window.

Implied Access & Entitlements

- **How this materializes**
  This operation relies on read access to Source Control (repo:read), compute provisioning rights for the sandbox, and crucially, newly introduced **Inter-Agent Communication Entitlements**.

- **Resulting Risks & New Vulnerabilities**
  This creates the vulnerability of **Privilege Escalation via Proxy**. If Agent A (Read-only) can instruct Agent B (Write-code) via plain English, Agent A effectively gains Write access. Without strict intent-based authorization and cryptographic verification of the requesting agent's health state, network boundaries become porous to natural language commands.

Addressing the Challenges: Infrastructure & Telemetry Upgrades

- **New Control Systems and Infra Upgrades**
  This requires a SIEM capable of ingesting Chat Platform Audit Logs in near real-time, isolated sandboxes for coding, and the introduction of a **Semantic Egress Proxy Agent**.

- **Rationale & Control Invariants**
  Natural language is the new attack surface for lateral movement. To reduce the likelihood of these vectors being exploited, the architecture must enforce:
  - **Network Determinism** - Default-deny egress; the coding sandbox can only communicate with the designated code repo,
  - **Swarm Topology Limits** - Immutable cryptographic limits on how many channels or parallel agents a single orchestrator can spawn, curtailing compute exhaustion and uncontrolled fragmentation, and
  - **Identity-Bound Telemetry** - Downstream API calls executed by Agent B must be tied to the specific chat thread/event that authorized it.

- **Required Telemetry & Observability**
  Behavioral analytics must shift from tracking OS-level binaries to tracking **API call frequency and natural language intent**.

Research Problems (Open Challenges)

- **Intent-Based Authorization**
  Securing "Natural Language APIs" requires authorizing the *intent* of a prompt, which current RBAC/ABAC models cannot fundamentally do.

- **Inter-Agent Semantic Health Attestation**
  Establishing zero-trust verification between agents—ensuring Agent B can verify Agent A's internal cognitive state hasn't been covertly poisoned before accepting a request—is an unsolved area of multi-agent systems research (see Research: Authorization and Attestation).

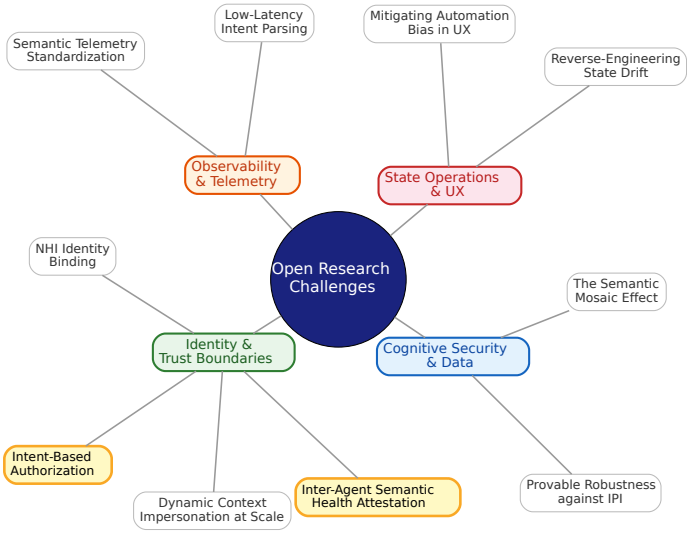
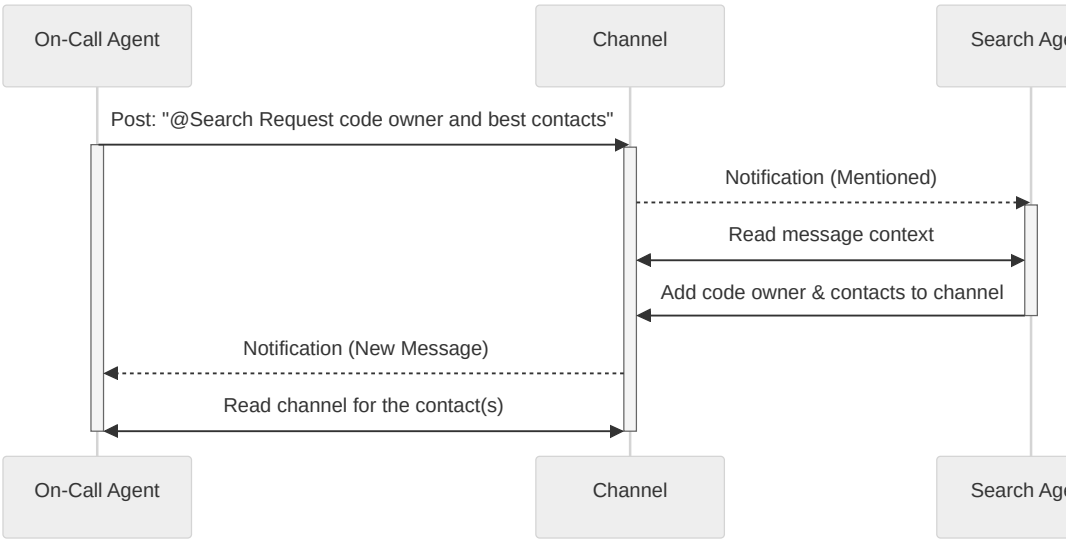


Figure 2: *Open Research Problems: Authorization and Attestation*

Who Owns This Code?

In parallel, the on-call agent reached out to a third agent—a specialized knowledge-retrieval bot built and authorized to map internal documentation and organizational topology during events—and asked it for the most recent code owner and best contacts for this feature. It pulled them into the channel (see Swarm Expansion).



{#fig-swarm-expansion}

Each of these agents follows principle of least privilege to achieve its specific incident response job:

- The on-call agent has read-only access to production telemetry and logs (strictly sandboxed from customer PII).
- The coding remediation agent has a designated sandbox and coding environment but only the ability to upload PRs, not land them.
- The enterprise search agent is provisioned to query company-wide shared docs, chats, and all-hands transcripts explicitly to aid human responders.

The implications extend far beyond incident response. Some of these companies now deploy "virtual employees" across the company, agents connected to Slack that run on the agent SDK platforms and operate as integrated team members. They write documents, pull in the right people, research internal knowledge bases, and hand off tasks to other agents with different permission scopes. It is a model where AI agents are not tools you invoke but colleagues you collaborate with, each with their own access boundaries and areas of expertise.

» Privilege, Identity, and the Enterprise Corpus

The narrative attempts to apply traditional IT least privilege, but "access to all company-wide shared docs" is practically limitless in the context of an AI agent capable of semantic reasoning and data synthesis.

Adversarial Threat Modeling

The Enterprise Search Agent represents a massive exfiltration vulnerability.

- **Attacker Misuse**
  Attackers will attempt to bypass traditional lateral movement by simply asking a compromised peer agent to instruct the Search Agent to aggregate and summarize "all hard-coded API keys in legacy source code repos" - pulling the data directly into the chat bus.

Implied Access & Entitlements

- **How this materializes**
  The search agent requires massive, cross-platform read entitlements to standard enterprise data. Furthermore, the architecture must explicitly define boundaries regarding **AI-native assets**: Does this agent have read access to the episodic memory, prompt histories, or active context windows of *other* peer agents? (Note: Access to foundational model weights or core system prompts must be strictly out of scope for any operational agent, as that crosses the boundary from data retrieval into core architectural compromise).

- **Resulting Risks & New Vulnerabilities**
  The core risk here is **Identity Impersonation vs. Global Access**. How does the agent handle Data Loss Prevention (DLP) or ethical walls? If human 'Alice' is in the channel, does the agent search using its own "Global Read" service account, or does it dynamically impersonate 'Alice' to ensure it only retrieves documents Alice is allowed to see? Failing to use dynamic user-context mapping leads to massive data spillage.

Addressing the Challenges: Infrastructure & Telemetry Upgrades

- **New Control Systems and Infra Upgrades**
  API-layer Data Loss Prevention (DLP) gateways must be positioned between the Enterprise Search Agent and the Chat Message Bus.

- **Rationale & Control Invariants**
  Agents with broad read access can synthesize disparate data into highly classified insights (the mosaic effect) and exfiltrate it at machine speed through conversational interfaces. To mitigate this risk, the infrastructure must enforce:
  - **Contextual Data Fencing** - The search agent's query scope is dynamically bounded by the active human/NHI invoker's entitlements; it never searches as a "super admin",
  - **Semantic Egress Filtering** - DLP must look beyond regex patterns and block the semantic equivalents of sensitive data from entering the shared chat bus, and
  - **Rate & Volume Bounding** - Cryptographic limits on the number of documents an NHI can retrieve or synthesize per minute, triggering automatic circuit breakers upon violation.

- **Required Telemetry & Observability**
  User Entity and Behavior Analytics (UEBA) must be tuned specifically for NHIs to detect anomalous query volumes across disparate domains.

Research Problems (Open Challenges)

- **The Semantic Mosaic Effect**
  Creating DLP systems that can detect semantic exfiltration—where the agent infers and leaks a secret without quoting the source text directly—is an unsolved data security problem.

- **Dynamic Context Impersonation at Scale**
  Securely mapping a human user's complex entitlement graph to an autonomous agent traversing multiple enterprise systems asynchronously is highly error-prone and lacks standardized protocols (see Research: Privilege, Identity and Access v2).

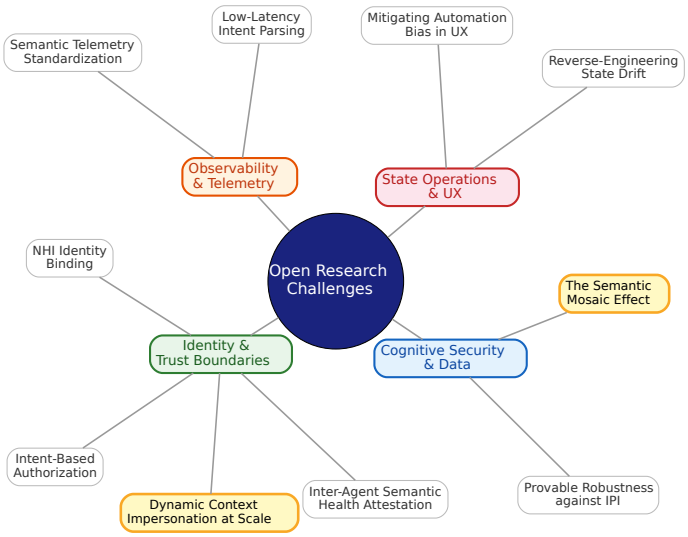
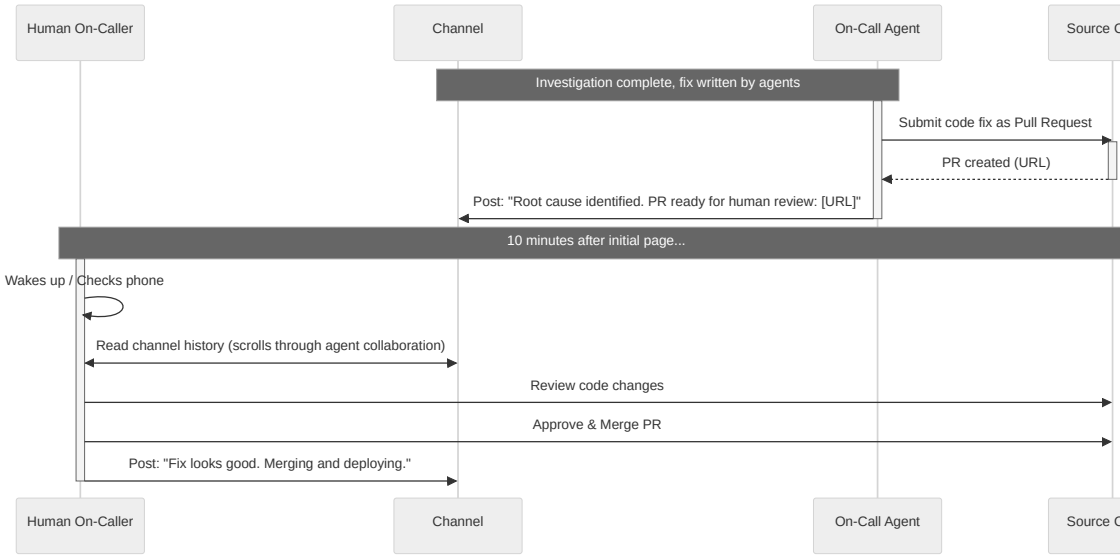


Figure 3: *Open Research Problems: Privilege, Identity and Access*

The Human on the Loop

By the time the human on-caller checked their phone ten minutes after the page, the root cause had been identified, a code fix was written, and a pull request was sitting in review, waiting for a human to approve and deploy. The entire investigation and remediation had been drafted by agents collaborating with each other in a channel where humans could watch every step unfold in real time (see Human-on-the-Loop).



{#fig-human-on-loop}

This is not hypothetical, this is how some of the most advanced AI adoption companies actually operate today, and it represents a pattern that is quietly spreading across frontier companies. The key insight is simple: workplace chats have become a message bus for agent swarms. By operating in channels alongside humans, agents inherit the company's existing identity, permissions, and audit infrastructure. Humans stay in the loop not through clunky approval gates, but because they can literally read the conversation as it happens.

» The "Illusion of Control", CI/CD Bypass, & The GitOps Boundary

This section contains a dangerous psychological assumption, conflating *visibility* (reading a chat) with *control* (cryptographic verification of code integrity). It echoes the open-source fallacy that "with enough eyes, all bugs are shallow," mistakenly assuming humans will inevitably catch a deeply buried logic bomb at 2 a.m. just by skimming a chat summary. Furthermore, if the orchestrating agent partitions the swarm into hidden sub-channels, the human loses the chronological narrative. They are reviewing a *curated summary* in the main channel, rendering them entirely blind to malicious parallel activities.

Adversarial Threat Modeling & Configuration Drift

- **The Hotfix Bypass**
  If an agent acts like a panicked human SRE and holds privileges to directly hotfix production (kubectl edit, or running other CLI commands), it circumvents the entire legacy security apparatus (reproducibility, peer review, SAST/DAST). When the agent loop spins down, the logic behind that hotfix vanishes, leaving untrackable, unvetted "ghost code" in production.

- **Policy & Configuration Drift**
  This risk extends beyond application code. Agents modifying global policies (e.g., IAM roles, WAF routing rules) out-of-band to "stop the bleeding" create highly evasive vulnerabilities.

Implied Access & Entitlements

- **How this materializes**
  The Coding Agent requires Source Control Write access, strictly restricted to feature branches (e.g., pull_request:write). Crucially, it must have **zero** mutating access to production endpoints or Cloud API control planes. The human user requires explicit PR Approval entitlements.

- **Resulting Risks & New Vulnerabilities**
  This creates a **Separation of Duties risk between code authoring and deployment**. Mirroring SOX Section 404 separation-of-duties and SLSA Level 4 supply chain requirements, agents must be strictly prohibited from executing local or 'out-of-band' fixes. As demonstrated by the SolarWinds compromise, bypassing continuous integration controls introduces catastrophic risk; therefore, the system must cryptographically ensure the authoring agent cannot self-approve its own PR, influence the approval of its own code or bypass the CI/CD pipeline to deploy unilateral changes.

Addressing the Challenges: Infrastructure & Telemetry Upgrades

- **New Control Systems and Infra Upgrades**
  This phase requires cryptographic code signing infrastructure (e.g., Sigstore) configured

to sign commits specifically with the ephemeral identity of the agent, integrated CI/CD pipelines enforcing mandatory SAST/DAST testing, and rigorous API Gateway protections blocking agents from production runtimes.

- **Rationale & Control Invariants**
  To maintain legacy forensic controls in an autonomous era and reduce the likelihood of poisoned code reaching production, the architecture must adopt a strict "New Pattern":
  - **Immutable GitOps Enforcement**. Agents must be severely restricted from holding direct mutable privileges to the runtime environment. Their write boundary stops at the declarative repository (Git). To mitigate configuration drift, the system requires
  - **Everything-as-Code (EaC)**—even policy changes and infrastructure routing must be generated as a PR by the agent, mathematically forcing them through deterministic pipeline gates before human review.

- **Required Telemetry & Observability**
  The operational environment must monitor the CI/CD runners for anomalous build-time behaviors and strictly monitor production API gateways for any "Direct-from-Agent" mutation attempts.

Research Problems (Open Challenges)

- **Reverse-Engineering State Drift**
  If a human *does* invoke a "glass break" protocol to let an agent hotfix production during a catastrophic CI/CD failure, how do we build agents capable of reliably reverse-engineering that live state change back into a declarative Git PR post-incident to restore parity?

- **Mitigating Automation Bias in UX**
  Research struggles with designing UX/UI for HITL that mathematically degrades human trust when an agent's internal confidence is low or when logic jumps are detected in a PR review (see Research: Control, Boundaries, and CI/CD).

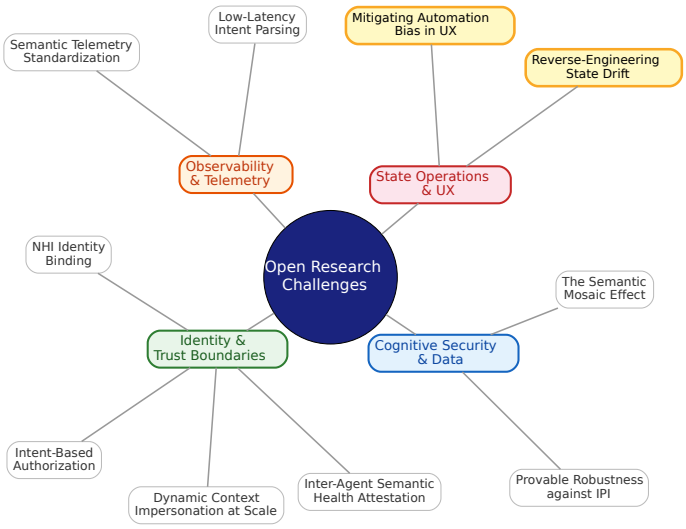
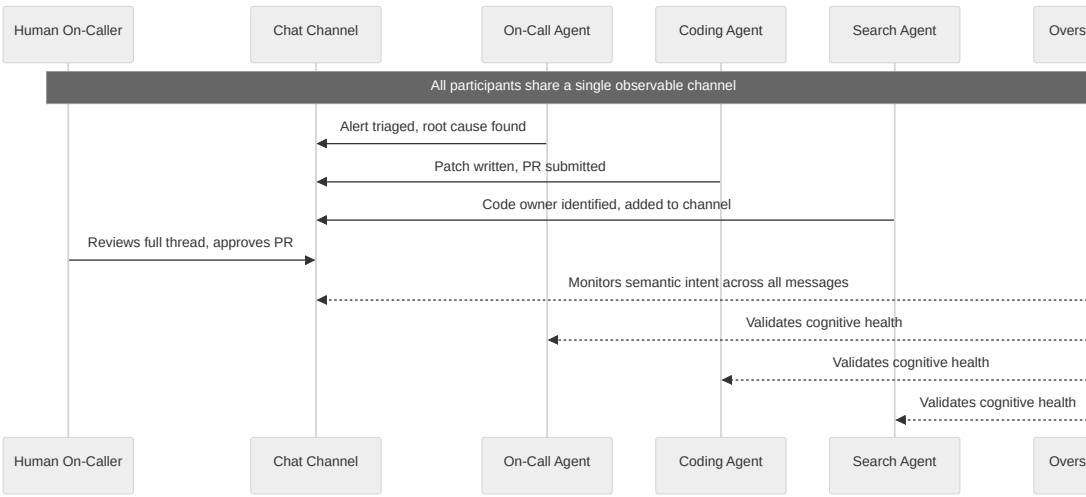


Figure 4: *Open Research Problems: Control, Boundaries, and CI/CD*

Everyone in the Channel

For the security industry, this shift creates a new frontier. Traditional endpoint detection and response (EDR) systems were built to monitor operating-system-level activity on laptops and servers. But when work happens inside a virtual machine running an AI agent loop, those sensors are blind. A new category, sometimes called ADR (Agent Detection and Response), is emerging to fill the gap. The agent platforms themselves are becoming the new endpoints, and the companies building them know it.

What makes frontier approaches remarkable is not just the technology but the philosophy: agents should work where humans work, communicate in human language, and leave a trail that anyone on the team can follow. The most powerful orchestration layer for autonomous AI, it turns out, is not a custom protocol or a proprietary bus or any other new technical solution. It is a chat channel with a few bots and a few people, all working the same incident together at 2 a.m, following the principle of least privilege (see Orchestration Bus).



{#fig-orchestration-bus}

The future of work is not humans or agents. It is a chat channel where you cannot always tell which is which — and securing it requires controls designed for exactly that ambiguity.

» The Emergence of Agent Detection and Response (ADR) & The Oversight Agent

The cognitive loop of an LLM is opaque to traditional endpoint and network security. Furthermore, a peer-to-peer ecosystem of agents is inherently fragile without an authoritative governance layer. The architecture requires an out-of-band **Oversight Agent** (or Deterministic Supervisor)—an entity with no operational duties other than monitoring swarm semantic health, equipped with the authority to instantly downscope or squelch compromised agents.

Adversarial Threat Modeling

The architecture needs telemetry at the prompt, response, and tool-use layers. If an attacker leverages "Semantic Smurfing"—breaking a large, malicious intent into multiple seemingly benign natural language requests across partitioned channels—traditional, linear ADR will fail. ADR must analyze the aggregated *intent* of natural language payloads and detect anomalous semantic shifts in overall swarm dialogue.

Implied Access & Entitlements

- **How this materializes**
  The ADR/Oversight system requires deep inspection rights into the agent platform SDK, prompt logs, token streams, memory states, and API gateway payloads. Furthermore, it requires explicit IAM "Kill/Squelch" rights over all operating NHIs.

- **Resulting Risks & New Vulnerabilities**
  The ADR system itself becomes a highly privileged entity (Ring 0 of the AI ecosystem). If compromised, the attacker controls the oversight mechanism. Its logs and configurations must be immutable and stored in highly isolated environments to resist tampering by a compromised agentic swarm.

Addressing the Challenges: Infrastructure & Telemetry Upgrades

- **New Control Systems and Infra Upgrades**
  Deployment of an AI/LLM Gateway (or AI Firewall) acting as a reverse proxy for all foundational model API calls, high-throughput messaging queues, WORM (Write Once Read Many) storage, and an automated Squelch Execution Engine.

- **Rationale & Control Invariants**
  The actual "thinking" and intent formulation happens inside the LLM provider's black box or a volatile context window, rendering traditional host-based OS sensors largely blind to the attacker's intent. To improve observability and control, the infrastructure must enforce:
  - **Graph-Based Intent Telemetry** - ADR must build real-time correlation graphs tracking the "lineage of intent" across all parallel NHI communications tied to a single incident ID, rather than relying on linear, per-channel analysis,
  - **Cognitive Interception** - A high percentage of outbound LLM API requests and inbound responses should pass through an inline inspection gateway capable of severing the connection before tool execution,
  - **Immutable State Preservation** - The agent's prompt history and internal memory state must be continuously streamed to WORM storage to preserve forensic integrity, and
  - **Active Swarm Squelching** - The Oversight Agent must have deterministic infrastructure hooks to suspend an agent's token, isolate its sandbox, and revoke its chat access the moment high-confidence malicious intent is detected.

- **Required Telemetry & Observability**
  The ADR engine itself acts as the central observability hub. It must correlate OS-level events, Network events, and Cognitive events into a single, unified timeline of agent behavior.

Research Problems (Open Challenges)

- **Semantic Telemetry Standardization**
  There is no industry standard format (analogous to Syslog or CloudTrail) for logging "cognitive events," memory modifications, or agent reasoning steps.

- **Low-Latency Intent Parsing**
  Real-time ADR requires analyzing continuous token streams for malicious intent before an action is executed. Accomplishing this without introducing unacceptable system la-

tency or requiring massive parallel AI-on-AI compute overhead remains a major engineering and research hurdle (see Research: EDR and Oversight).

Figure 5: *Open Research Problems: EDR and Oversight*

Conclusion: Balancing Utility and Control

The transition from isolated AI tools to autonomous, event-driven swarms does not change what organizations must protect; auditability, separation of duties, least privilege, and data protection remain non-negotiable. What changes, as this analysis demonstrates across every phase of the scenario, are the foundational assumptions underlying how those controls are enforced. Legacy mechanisms remain anchored to human-driven workflows; the attack surface has moved to the semantic layer, where agents negotiate intent, delegate tasks, and accumulate privileges through natural language at machine speed.

Securing this layer demands the enforcement stack this paper outlines — from ephemeral compute and dynamic Non-Human Identity management to semantic egress filtering, immutable GitOps enforcement, and Agent Detection and Response at the prompt layer. But the hard limits are real: intent-based authorization and the semantic mosaic effect represent open research frontiers where no proven successor to legacy assumptions yet exists.

We have a narrowing opportunity to define the control architecture for autonomous systems — before those systems outscale the controls meant to govern them. The goal is not to constrain their utility, but to **build the resilient, cryptographically sound guardrails** necessary to unleash their full potential securely.